

# TBC-ac4C: Identification of N4-acetylcytidine sites by incorporating TCN and BiGRU based on multi-head cross-attention

\*\*\*  
\*\*\*  
\*\*\*  
\*\*\*  
\*\*\*

\*\*\*  
\*\*\*  
\*\*\*  
\*\*\*  
\*\*\*

\*\*\*  
\*\*\*  
\*\*\*  
\*\*\*  
\*\*\*

**Abstract**—N4-acetylcytidine (ac4C), a pivotal post-transcriptional modification, modulates mRNA lifecycle dynamics and is linked to numerous pathologies. Accurate genome-wide identification of ac4C sites is fundamental to functional characterization, but existing in-silico methods require substantial improvement. To address this, we develop TBC-ac4C method, which integrates the transformer encoder with the parallel temporal convolutional network (TCN) architecture and bidirectional gated recurrent unit (BiGRU), capturing comprehensive spectrum of sequence features from global dependencies to local hierarchical patterns. These features are synergistically fused via multi-head cross-attention mechanism to generate highly discriminative representation for prediction. In the independent test set, TBC-ac4C achieves ACC, MCC, and AUC gains of 3.26%, 0.0658, and 0.0361, respectively, relative to the current state-of-the-art method STM-ac4C. Further generalization tests validate the model’s stability on the balanced and imbalanced datasets, where it consistently outstanding, particularly under imbalanced conditions (Sn is 89.29%, AUC is 0.8962). TBC-ac4C represents a robust and effective tool that will facilitate ac4C-related functional genomics exploration. The datasets and codes are publicly available at <https://github.com/xinyizhang186/TBC-ac4C>.

**Keywords**—N4-acetylcytidine sites, temporal convolutional network, bidirectional gated recurrent unit, multi-head cross-attention mechanism

## I. INTRODUCTION

RNA chemical modifications constitute a critical layer of post-transcriptional control, with over 170 distinct types across tRNA, rRNA, and mRNA [1]. Unlike canonical epigenetic marks, they dynamically modulate mRNA stability, transcription, and translation to directly impact cellular functions and disease pathophysiology [2-5]. Among these, N4-acetylcytidine (ac4C) has attracted substantial attention for its high evolutionary conservation, chemical stability, and broad functional relevance [6-7]. Initially characterized in tRNA and rRNA, ac4C sites have recently been revealed in mRNA by high-throughput sequencing [8-9]. Catalyzed by acetyltransferase NAT10, ac4C is established as a conserved regulatory hub across species and pathological contexts [10,11]. Its dysregulation contributes to human pathologies including cancer and metabolic disorders [12,13]. In plants, ac4C modulates growth and development via mRNA modification, as demonstrated in *Arabidopsis thaliana*, tobacco, and maize [14-16]. Consequently, precise detection of ac4C sites is a fundamental prerequisite for developing novel therapeutics and enhancing crop genetics.

Advances in high-throughput sequencing and machine learning have positioned computational prediction as a robust

alternative for ac4C site identification. In 2019, Zhao et al. pioneered this domain with PACES [17] which employed a random forest using position-specific dinucleotide sequence profile (PSDSP) and k-nucleotide frequency (KNF) features. In 2020, Alam et al. proposed XG-ac4C [18], integrating trinucleotide pseudo electron-ion interaction potential (PseEIIP) and electron-ion interaction pseudopotentials (EIIP) with XGBoost to enhance prediction accuracy. In 2021, Wang et al. introduced DeepAc4C [19], which utilized the convolutional neural network (CNN) to fuse physicochemical patterns with nucleic acid distribution features. In 2022, Su et al. engineered iRNA-ac4C [20] to combine k-mer, nucleotide chemical property (NCP), and accumulated nucleotide frequency (ANF) encodings with gradient boosting decision tree (GBDT), outperforming prior models.

The computational identification of ac4C sites has been significantly advanced by the development of sophisticated deep learning. In 2023, Lai et al. introduced LSA-ac4C [21], a framework based on two-layer LSTM and self-attention mechanism, and concurrently developed Auto-ac4C [21], which leveraged transformer for automated feature engineering. That same year, Jia et al. build DLC-ac4C [22], which integrated DenseNet and BiLSTM, achieving an accuracy of 82.97%. The year 2024 witnessed an explosion of advanced models. ac4C-AFL [23] constructed an optimized ensemble predictor from 176 baseline models; DPNN-ac4C [24] adopted a dual-path neural network of CNN and BiGRU to capture local and global dependencies; NBCR-ac4C [25] utilized nucleotide transformer and DNABERT2 for context-aware embeddings, while MetaAc4C [26] combined the pre-trained BERT with WGAN-GP; STM-ac4C [27] focused on multi-scale feature extraction using selective kernel convolution (SKC) and temporal convolutional network (TCN); Voting-ac4C [28] integrated the transformer-based RNAErnie pre-trained large model with six traditional feature encoding methods (One-hot, ENAC, C2, ND, TPCP, Ksnpf), utilizing a deep neural network for feature dimension reduction and soft voting ensemble learning to improve ac4C site prediction performance. This progression continued in 2025 with Liu et al.’s TransCNN [29], which fused transformer and TextCNN modules to enhance predictive performance. Despite these advancements, substantial room for improvement remains. Therefore, there remains a critical need for more accurate, robust, and efficient computational frameworks to deepen our understanding of ac4C biology.

We present TBC-ac4C, an advanced deep learning architecture for superior ac4C site prediction. Its core contribution is a multi-branch architecture that employs a transformer encoder for global dependencies, with parallel TCN and BiGRU branches to extract hierarchical and

contextual features, respectively. To synergize these pathways, we introduce the multi-head cross-attention mechanism that adaptively fuses the heterogeneous features, creating a highly discriminative representation. Critically, comprehensive evaluation on both balanced and imbalanced datasets confirms the model's robustness and generalizability, with performance metrics surpassing existing approaches and overcoming previous methodological constraints.

## II. MATERIALS AND METHODS

### A. Datasets

The current work adopted three datasets to develop and validate our model: a primary benchmark dataset for training and two supplementary datasets for evaluating robustness and generalization. The benchmark dataset was constructed based on the work of Su et al. [20], using acRIP-seq data from Arango et al. [5] that detected 4250 modification ac4C. Positive samples consisted of 201 bp sequences centered on the cytosine sites within these peaks. Negative samples, of identical length, were generated from randomly selected, non-ac4C modified genomic regions also centered on the cytosine. To reduce homology bias, CD-HIT [30] was used to remove sequences at an 0.8 identity threshold. The resulting non-redundant dataset contained 2758 sequences (1379 positive and 1379 negative), which was partitioned at a 4:1 ratio into the training set (2206 sequences) and the independent test set (552 sequences). Furthermore, two additional validation datasets from MetaAc4C [26] were incorporated, but processed with a stricter CD-HIT [30] threshold (0.4 identity) to avoid overestimation. The sequences in these datasets featured a length of 415 bp with the ac4C site centrally located. That included a balanced dataset and an imbalanced dataset with a 5:1 negative-to-positive sample ratio, both sharing a common independent test set. Detailed statistics for all datasets are presented in TABLE I.

### B. Transformer-based sequence feature encoding

We employ the transformer encoder block to extract numerical feature vectors from RNA sequences. Adapted from Vaswani et al.'s transformer model [31], this approach effectively captures long-range dependencies in biological sequences. The encoder block comprises multi-head self-attention mechanism and feed-forward network, with layer normalization applied to ensure training stability.

The core of the encoder is the multi-head self-attention mechanism, which calculates the relationships between all nucleotide pairs in a sequence. It employs scaled dot-product attention, where Query (Q), Key (K), and Value (V) matrices calculate attention weights, as shown in (1). The attention head outputs are aggregated via concatenation to generate comprehensive features representation in (2).

$$Q_i = XW_i^Q; K_i = XW_i^K; V_i = XW_i^V$$

$$head_i = Self\_Attention(Q_i, K_i, V_i) = \text{soft max} \left( \frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i \quad (1)$$

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_h) \quad (2)$$

where  $W_i^Q, W_i^K$  and  $W_i^V$  are weight matrices for the  $i$ -th head, and  $d_k$  represents the dimension of the key vectors  $K_i$ .

To incorporate sequential order, which the transformer itself does not process, we integrate the positional encoding.

This method generates a unique vector for each position using sine and cosine functions in (3). This position encodings are then element-wise fused to the input embeddings.

$$PE_{(pos, 2i)} = \sin \left( \frac{pos}{10000^{2i/d}} \right) \quad (3)$$

$$PE_{(pos, 2i+1)} = \cos \left( \frac{pos}{10000^{2i+1/d}} \right)$$

where  $pos$  represents the position of an element in the sequence, and  $d$  is the hidden state dimensionality.

### C. Model framework

This paper proposes the TBC-ac4C model to address the identification of ac4C sites. Its architecture integrates four core modules, as illustrated in Fig. 1. (1) Feature encoding: a transformer encoder block converts RNA sequences into contextual embeddings. (2) Parallel feature extraction: the encoded features are processed in parallel by temporal convolutional network (TCN) and bidirectional gated recurrent unit (BiGRU) to temporal dependencies and capture long-range spatial correlations, respectively. (3) Attention fusion: multi-head cross-attention mechanism is employed to adaptively weight and integrate the output representations from the TCN and BiGRU, thereby generating an enhanced feature representation. (4) Dimensionality reduction and classification: the fused high-dimensional features undergo max pooling along the sequence length dimension to enhance the model's generalization capability and reduce computational overhead. Finally, a two-layer multi-layer perceptron (MLP) is used to output the prediction results.

#### 1) TCN module

Temporal convolutional network (TCN) [32] is incorporated as a parallel branch to extract hierarchical features. In contrast to recurrent neural network (RNN) [33], TCN is combined with causal convolution, dilated convolution, and residual connection. This design expands the receptive field while preserving temporal causality, thereby effectively capturing long-range dependencies.

##### a. Causal convolution

The causal convolution ensures that the output at any position is derived solely from inputs at or before that position. This structure prevents any leakage of information from subsequent positions. To maintain consistent sequence length across layers, zero-padding is applied. The procedure is formally expressed as:

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-i} + b \quad (4)$$

where  $w_i$  is the convolution kernel weight,  $b$  is the bias term, and  $k$  is the kernel size.

##### b. Dilated convolution

To efficiently model long-range dependencies, TCN employs dilated convolutions. By stacking layers of causal dilated convolutions, the receptive field of the network grows exponentially with its depth, enabling TCN to process exceptionally long sequences using a comparatively shallow architecture. The mathematical formula is as follows:

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-d \cdot i} + b \quad (5)$$

where  $d$  is the dilation factor. When  $d=1$ , the dilated causal convolution degenerates into a standard causal convolution.

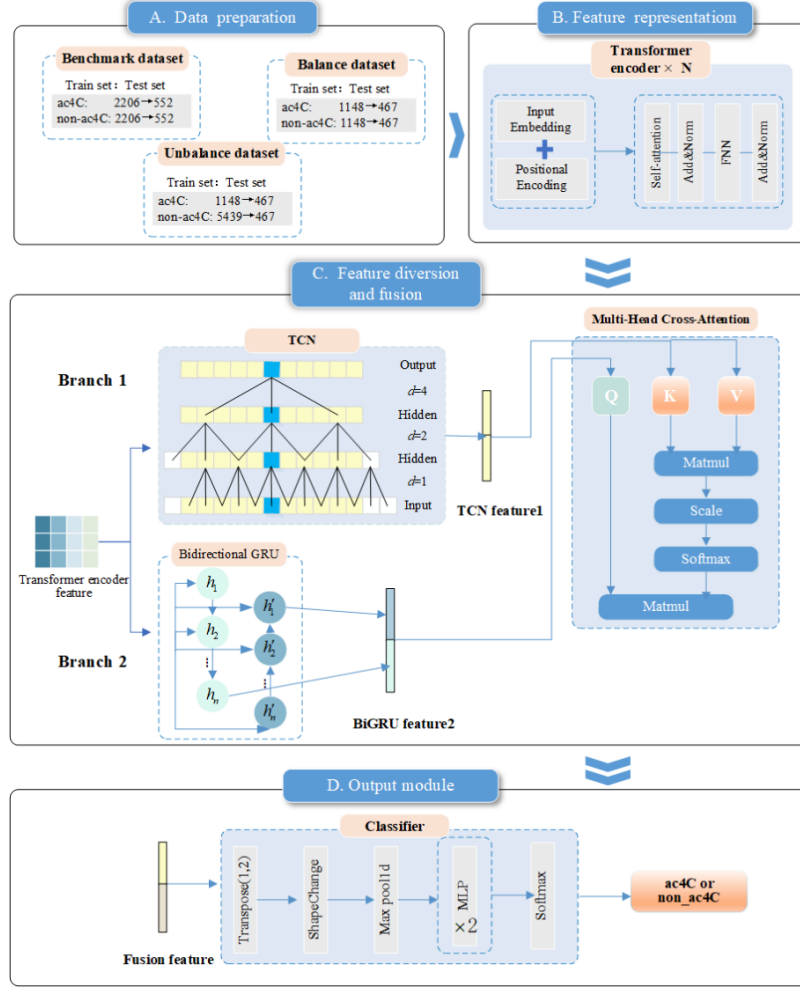


Fig. 1. The flowchart of the TBC-ac4C model.

TABLE I. DETAILS OF THE DATASETS

|                          | Train-Test | ac4C | Non-ac4C | Total |
|--------------------------|------------|------|----------|-------|
| <b>Benchmark Dataset</b> | Train      | 2206 | 2206     | 5516  |
|                          | Test       | 552  | 552      |       |
| <b>Balance Dataset</b>   | Train      | 1148 | 1148     | 3230  |
|                          | Test       | 467  | 467      |       |
| <b>Unbalance Dataset</b> | Train      | 1148 | 5439     | 7521  |

### c. Residual block

TCN achieves stable training of deep networks via stacked residual blocks. Every residual unit comprises a pair of dilated causal convolutional layers, with an interspersed activation function, weight normalization, and a dropout layer to enhance non-linear capacity and accelerate convergence. A  $1 \times 1$  convolution aligns input and output dimensions for residual connections—shortcuts that ensure unobstructed information flow to alleviate the degradation issues prevalent. This design maintains training stability and enhances long-term dependency capture.

### 2) BiGRU module

The bidirectional gated recurrent unit (BiGRU) [34] is employed to further capture sequential context from the RNA embeddings. The GRU architecture uses update and reset gates to regulate information propagation, enabling the effective modeling of long-range dependencies. By processing sequences bidirectionally, the BiGRU extracts both forward and backward contextual features from the time

series. The core operations of the GRU are defined according to the following relations:

$$\begin{aligned}
 z_t &= \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \\
 r_t &= \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \\
 \tilde{h}_t &= \tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t] + b_h) \\
 h_t &= (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t
 \end{aligned} \tag{6}$$

where  $z_t$  and  $r_t$  represent the update and reset gates, respectively.  $\tilde{h}_t$  is the candidate hidden state, while  $h_{t-1}$  and  $h_t$  are the hidden states at the previous and current time steps, respectively.  $\sigma$  represents the sigmoid activation function.

Consequently, the hidden states computed by the forward and backward GRU components are formally defined as:

$$\begin{aligned}
 \tilde{h}_t &= GRU(x_t, \tilde{h}_{t-1}) \\
 \tilde{h}_t &= GRU(x_t, \tilde{h}_{t-1})
 \end{aligned} \tag{7}$$

At time step  $t$ , the final output state of the BiGRU hidden layer,  $h_t$  is obtained by integrating these two hidden states through an activation function  $f$ , as shown below:

$$h_t = f(W_{\tilde{h}_t} \cdot \tilde{h}_t + W_{\tilde{h}_t} \cdot \tilde{h}_t + b_t) \quad (8)$$

### 3) Attention fusion model

In contrast to the self-attention mechanism, which models intra-sequence dependencies, the cross-attention mechanism [35] is specifically designed to model the relationships between different sequences. Its core principle is to allow one sequence (referred to as the “query”) to attend to another sequence (the “key” and “value”), thereby enabling the fusion of semantics across sequences.

Cross-attention captures the interactive dependencies between different sequence data by computing a weighted representation of sequence B relative to sequence A. Other processes are consistent with those of the self-attention mechanism. The expression can be formally stated as:

$$Q_i = X_A W_i^Q; K_i = X_B W_i^K; V_i = X_B W_i^V$$

$$head_i = \text{Cross\_Attention}(Q_i, K_i, V_i) = \text{soft max} \left( \frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i \quad (9)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, head_2, \dots, head_h) W^O$$

Where  $X_A$  and  $X_B$  represent the input from sequence A and B, respectively.

### 4) Output model

The output module progressively transforms latent features  $X$  (derived from cross-attention) into class probabilities through two-layer multi-layer perceptron (MLP). The first layer employs a LeakyReLU activation function for nonlinear feature space compression, while the second layer uses a Softmax function to generate the final class probability distribution. The operations are as follows:

$$H_1 = \sigma(W_1 X + b_1), P = \text{Soft max}(W_2 H_1 + b_2)$$

$$\sigma(x) = \text{Leaky ReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases} \quad (10)$$

where  $W_1, W_2$  and  $b_1, b_2$  correspond to the learnable parameters: weights and biases, and  $\alpha$  controls the negative slope.

For the ac4C site identification (binary classification tasks), we optimize the cross-entropy loss with  $L_2$  regularization. The total loss function is formulated as:

$$L = \alpha [-y \log p(y) - (1-y) \log (1-p(y))] + \frac{\lambda}{2} \|W\|_2^2 \quad (11)$$

where  $y$  denotes the ground truth label,  $p(y)$  denotes the model's predicted probability distribution, and  $\lambda$  is the regularization coefficient.

### D. Performance metrics

In this paper, we adopt four core binary classification metrics: sensitivity (Sn), specificity (Sp), accuracy (ACC), and the Matthews correlation coefficient (MCC). Sn and Sp quantify the classifier's capacity to correctly discriminate positive and negative samples, respectively. ACC represents

the overall classification precision across all samples. MCC measures correlation between true and predicted classifications, offering robust evaluation even for imbalanced datasets. Furthermore, we utilized the receiver operating characteristic (ROC) curve and the precision-recall (PR) curve to visually and quantitatively assess the model's performance. The formulas are defined below:

$$Sn = \frac{TP}{TP + FN}$$

$$Sp = \frac{TN}{TN + FP}$$

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (12)$$

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FN) \cdot (TP + FP) \cdot (TN + FP) \cdot (TN + FN)}}$$

$$AUC \text{ or } auROC = \int_0^1 TPR(FPR^{-1}(x)) dx$$

$$auPRC = \int_0^1 Precision(Recall^{-1}(x)) dx$$

where  $TP$ ,  $FP$ ,  $TN$ , and  $FN$  refer to true positives, false positives, true negatives, and false negatives, respectively. AUC (auROC) and auPRC are quantified by the area under the ROC curve and the area under the PR curve, respectively.

## III. RESULTS AND DISCUSSION

### A. Experimental setting and hyperparameter optimization

All models are implemented in Python 3.7 and torch 1.13.1+cu116, and trained on the High-Performance Computing Platform at Xidian University. We use the Adam [36] optimizer for model training. The learning rate is dynamically adjusted according to validation performance via the ReduceLROnPlateau scheduler [37]. Models are trained with 64 for batch size and 100 for the maximum epochs. To prevent overfitting, we utilize the 10-fold cross-validation scheme coupled with the early stopping criterion: training is halted if the validation accuracy can't be improved by at least 0.01% over 15 consecutive epochs.

We perform automated hyperparameter optimization using the Optuna framework [38], which leverages the Bayesian optimization strategy [39]. The tuning process targeted three key hyperparameters: the embedding dimension, the number of TCN intermediate channels, and the convolutional kernel size, with their search spaces detailed in TABLE II. The optimal set of hyperparameters is identified after 30 optimization trials. The optimization results are visualized in Fig. 2. Fig. 2(A) tracks convergence to the optimal parameter set. Fig. 2(B) shows convolutional kernel size exerted the strongest performance impact, followed by embedding dimension, with TCN channel number least influential. Fig. 2(C) illustrates how hyperparameter values influence model performance to guide optimal range selection. The selected hyperparameters are 64 for embedding dimension, [64, 64, 100] for TCN intermediate channel sizes, and 5 for convolutional kernel size.

TABLE II. HYPERPARAMETER SEARCH SPACE OF TBC-AC4C MODEL

|                     | Searched values                           |
|---------------------|-------------------------------------------|
| Embedding dim       | [16, 32, 64]                              |
| TCN middle channels | [(64,64,100), (64,64,128), (128,128,256)] |
| TCN kernel size     | [3, 5, 7, 9]                              |

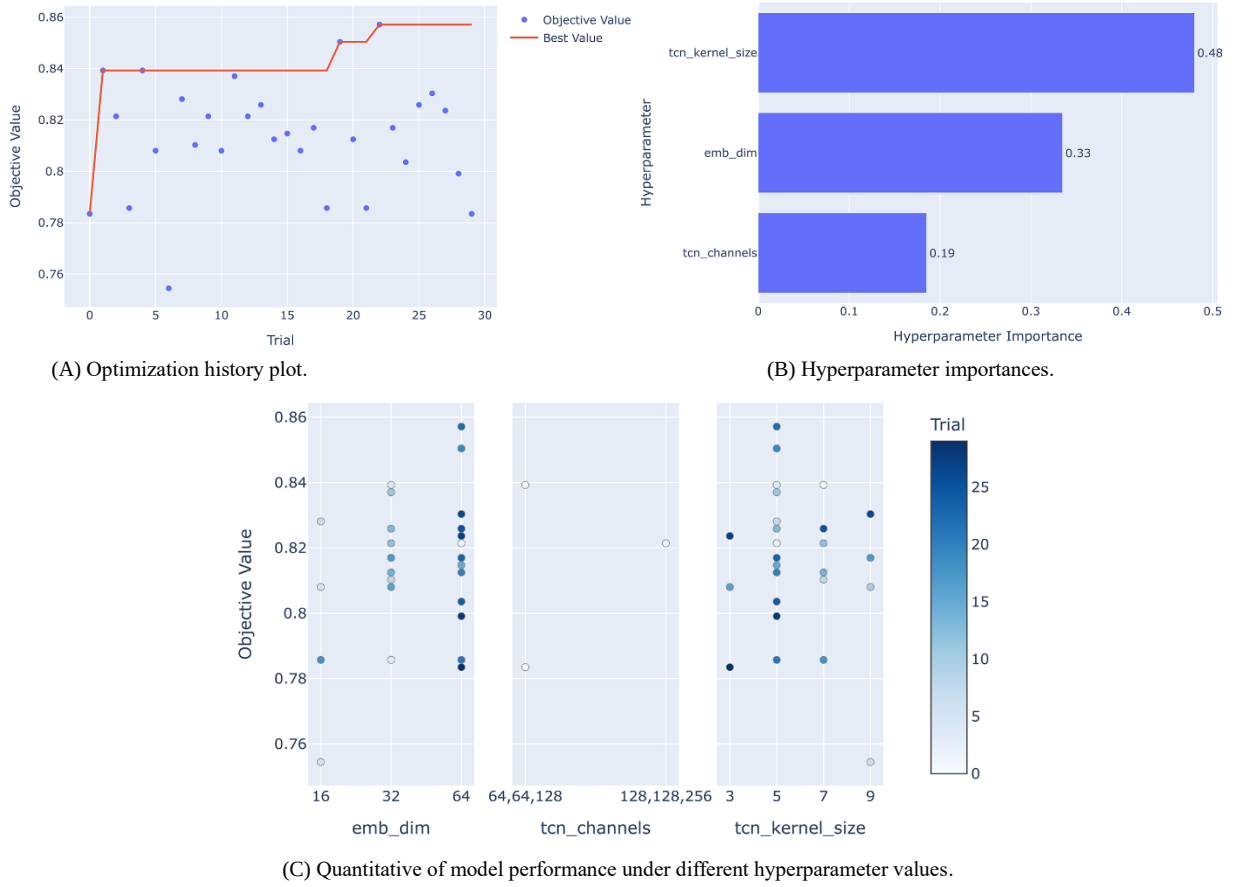


Fig. 2. Hyperparameter optimization results.

### B. Performance comparison with different models

The predictive power of TBC-ac4C is first evaluated by 10-fold cross-validation on the benchmark dataset. The detailed evaluation criteria encompassing ACC, Sn, Sp, and MCC are presented in TABLE III, with the corresponding ROC and PR curves shown in Fig. 3. It yields high average metrics: ACC =  $84.25\% \pm 3.15\%$ , Sn =  $87.99\% \pm 4.69\%$ , Sp =  $80.51\% \pm 5.77\%$ , MCC =  $0.6898 \pm 0.061$ , and AUC =  $0.9032 \pm 0.0221$ . Collectively, these results demonstrate the model's robust and stable performance, as evidenced by the low standard deviations across the validation folds.

To further assess its generalization and superiority, we benchmark TBC-ac4C against eleven state-of-the-art methods on the independent test set. These competing predictors included PACES [17], XG-ac4C [18], iRNA-ac4C [20], Auto-ac4C [21], LSA-ac4C [21], DLC-ac4C [22], STM-ac4C [27], DPNN-ac4C [24], ac4C-AFL[23], Voting-ac4C [28], and TransCNN [29]. As summarized in TABLE IV, TBC-ac4C outperforms all competitors, demonstrating superior results in four key metrics: ACC (88.04%), Sp (85.87%), MCC (0.7616), and AUC (0.9440). Notably, its MCC score is 0.0658 higher than the runner-up model named STM-ac4C. While XG-ac4C records the highest Sn at 92.57%, its low Sp of 59.78% indicates a high false-positive rate. In contrast, TBC-ac4C demonstrates more optimal balance across all evaluation metrics, underscoring its robust and superior predictive performance.

### C. Ablation experiments

To systematically dissect the contribution of each fundamental module to the overall performance of TBC-ac4C, TBC-ac4C model as the baseline, we conduct evaluation of

multiple model variants through both standardized benchmarking and independent test set validation: (i) a variant without the TCN module, (ii) a variant without the BiGRU module, and (iii) a variant where the cross-attention mechanism is substituted with self-attention. The comparative results are presented in Fig. 4. The outcomes clearly indicate that the fully integrated TBC-ac4C achieves the highest performance. Notably, the removal or replacement of any core modules led to a significant decline in accuracy, confirming the indispensable role of each component. More specifically, the TCN module emerges as the most critical contributor. In the independent test set, its removal causes the most pronounced performance degradation, with the ACC and MCC dropping by 4.16% and 0.0761, respectively. This underscores the TCN's essential function in effectively capturing long-range sequential dependencies. The impacts of removing the BiGRU layer or substituting the attention mechanism are less pronounced but still significant. Therefore, these results validate our decision to employ the complete, synergistic architecture as the final TBC-ac4C model.

TABLE III. THE PERFORMANCE OF TBC-AC4C IN EACH FOLD

|      | ACC(%)           | Sn(%)            | Sp(%)            | MCC                |
|------|------------------|------------------|------------------|--------------------|
| 1    | 82.58            | 87.33            | 77.83            | 0.6545             |
| 2    | 78.73            | 88.24            | 69.23            | 0.5853             |
| 3    | 82.31            | 88.24            | 76.36            | 0.6507             |
| 4    | 87.98            | 94.12            | 81.82            | 0.7653             |
| 5    | 85.03            | 83.71            | 86.36            | 0.7010             |
| 6    | 83.45            | 92.31            | 74.55            | 0.6795             |
| 7    | 87.98            | 89.55            | 86.43            | 0.7600             |
| 8    | 86.62            | 90.00            | 83.26            | 0.7342             |
| 9    | 86.85            | 89.09            | 84.62            | 0.7377             |
| 10   | 80.95            | 77.27            | 84.62            | 0.6206             |
| Mean | $84.25 \pm 3.15$ | $87.99 \pm 4.69$ | $80.51 \pm 5.77$ | $0.6898 \pm 0.061$ |

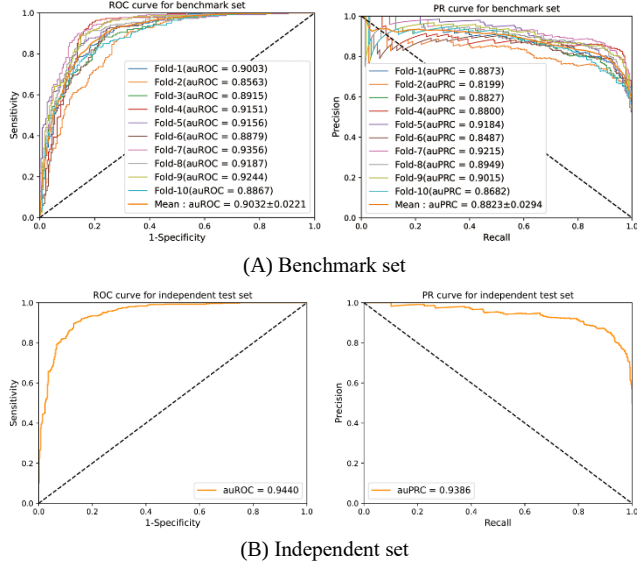


Fig. 3. ROC and PR curves for TBC-ac4C model.

TABLE IV. COMPARATIVE EVALUATION OF MULTIPLE METHODS ON THE INDEPENDENT TEST SET

|                 | ACC(%)       | Sn(%)        | Sp(%)        | MCC           | AUC           |
|-----------------|--------------|--------------|--------------|---------------|---------------|
| PACES           | 78.80        | 79.71        | 77.90        | 0.5762        | 0.8648        |
| XG-ac4C         | 76.18        | <b>92.57</b> | 59.78        | 0.5542        | 0.8713        |
| iRNA-ac4C       | 79.81        | 76.70        | 82.91        | 0.5970        | 0.8800        |
| Auto-ac4C       | 80.71        | 82.61        | 78.80        | 0.6146        | 0.8894        |
| LSA-ac4C        | 82.70        | 87.13        | 78.26        | 0.6566        | 0.8953        |
| DLC-ac4C        | 82.97        | 86.23        | 79.71        | 0.6608        | 0.9042        |
| STM-ac4C        | 84.78        | 85.87        | 83.70        | 0.6958        | 0.9079        |
| DPNN-ac4C       | 82.78        | 81.78        | 84.78        | 0.6578        | 0.9103        |
| ac4C-AFL        | 82.30        | 84.40        | 80.30        | 0.6470        | 0.8950        |
| Voting-ac4C     | 83.15        | 85.14        | 81.15        | 0.6635        | 0.8873        |
| TransCNN        | 82.89        | -            | -            | 0.6597        | 0.8925        |
| <b>TBC-ac4C</b> | <b>88.04</b> | <b>90.22</b> | <b>85.87</b> | <b>0.7616</b> | <b>0.9440</b> |

The bold font denotes the optimal value obtained in each column

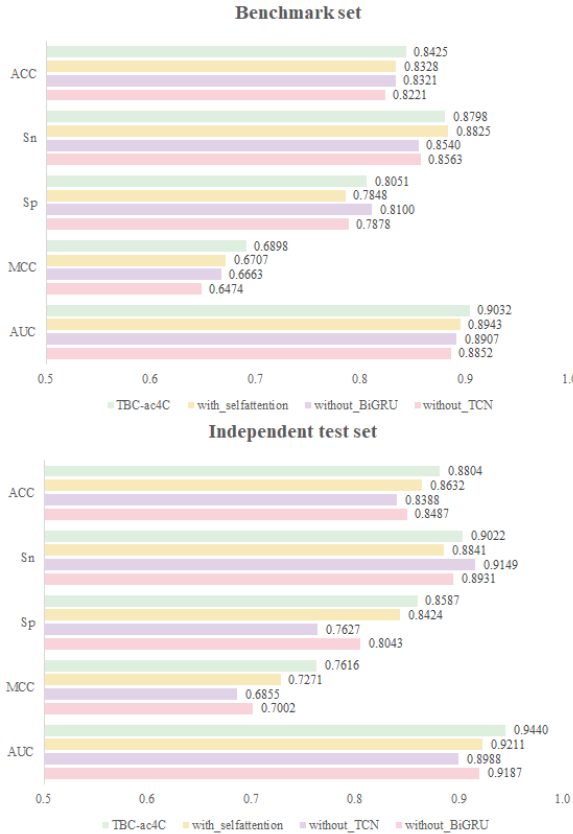


Fig. 4. Results of the ablation experiment for TBC-ac4C.

#### D. Visualization of feature dynamics

To elucidate the feature learning trajectory of the TBC-ac4C model, the high-dimensional feature representations acquired at distinct training stages are visualized. Using uniform manifold approximation and projection (UMAP) [40], we project the features extracted from the initial, intermediate (epoch 10), and final training stages into a 2D space. Fig. 5 demonstrates a clear progression of results. Initially, the feature embeddings of positive and negative samples are highly overlapping and largely indistinguishable. After just 10 epochs, noticeable separation between the two classes begins to form, indicating that the model is learning discriminative patterns. By the end of training, the samples resolve into two distinct and compact clusters with high inter-class separability. This demonstrates that the TBC-ac4C model successfully learns powerful and highly separable features, thereby confirming its effectiveness for this classification task.

#### E. Robustness and generalization analysis on balanced and imbalanced datasets

To evaluate the robustness and generalization of TBC-ac4C, we train and test the model on the balanced and imbalanced datasets from MetaAc4C [26], using a common independent test set for evaluation. The model's performance is benchmarked relative to reference under identical conditions, with detailed comparisons presented in TABLE V and TABLE VI. On the balanced dataset, TBC-ac4C exhibited superior performance, outperforming all competing models across four key metrics: 84.48% for ACC, 83.30% for Sn, 0.6897 for MCC, and 0.9080 for AUC. Especially, TBC-ac4C surpasses the runner-up model named STM-ac4C [27], with 2.47% higher ACC, 0.86% higher Sn, and 4.07% higher Sp, affirming its robustness on uniformly distributed data.

#### F. Discussion

This section synthesizes the above experimental findings and delves into their implications. In the independent test set, our model consistently surpasses state-of-the-art approaches across key evaluation metrics, underscoring the efficacy of its architecture. The contribution of each component within our framework is further substantiated through rigorous ablation studies, which confirms their essential role in achieving the model's superior performance. To ensure model interpretability, UMAP is employed for high-dimensional feature visualization. The results reveal that TBC-ac4C learns a highly discriminative feature space, establishing clear decision boundaries between ac4C and non-ac4C instances.

A clear separation between positive and negative samples and thereby providing a visual rationale for its predictive power. This strong learning capability underpins the model's robust generalization across diverse datasets, including both balanced and imbalanced scenarios. A critical finding from our evaluation on the imbalanced data, which mimics real-world biological conditions, is a strategic performance trade-off. While TBC-ac4C's ACC(82.98%) is marginally second to STM-ac4C (83.19%), it achieves the highest Sn (89.29%) and superior AUC (0.8962). This deliberate prioritization of sensitivity is a key features design, intended to minimize false negatives in biological discovery pipelines where the cost of a missed true site is significantly higher than that of a false positive.

Despite these promising results, we acknowledge avenues for future improvement. Our efforts will be directed toward: (i) refining the sensitivity-specificity balance through the



implementation of tunable modules; (ii) enhancing the predictive context by incorporating multi-modal data streams; and (iii) systematically evaluating the framework’s broader utility by extending its generalizability across diverse biological species and variant RNA epigenetic marks.

#### IV. CONCLUSIONS

This paper introduces TBC-ac4C, an integrated deep learning architecture designed for identifying ac4C sites in mRNA. Its hybrid architecture employs a transformer encoder to capture global sequence dependencies, which are then processed by parallel TCN and BiGRU branches to model temporal and contextual patterns, respectively. A pivotal cross-attention mechanism adaptively integrates these distinct feature sets to inform the final prediction. In the benchmark dataset, TBC-ac4C model consistently outperforms state-of-the-art methods. UMAP visualization demonstrates the framework’s ability to learn highly discriminative representations, while elucidating its underlying feature separation mechanism. Furthermore, the model exhibits exceptional generalization capabilities on both balanced and imbalanced datasets. From a biological perspective, this framework provides a powerful instrument to advance our understanding of the functional genomics of ac4C and explore its translational applications in precision medicine.

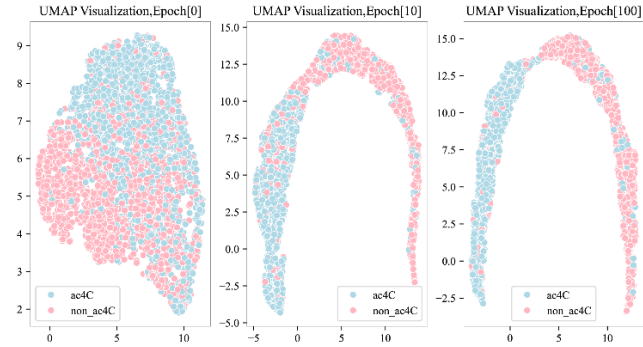


Fig. 5. Visualization of the training phase on the benchmark set using UMAP

TABLE V. COMPARATIVE EVALUATION OF MULTIPLE METHODSON THE BALANCED DATASET

|                | ACC(%)       | Sn(%)        | Sp(%)        | MCC           | AUC           |
|----------------|--------------|--------------|--------------|---------------|---------------|
| PACES          | 53.21        | 8.12         | <b>98.29</b> | 0.1484        | 0.5322        |
| XG-ac4C        | 76.33        | 58.23        | 94.43        | 0.5650        | 0.7633        |
| DeepAc4C       | 79.19        | 82.80        | 75.58        | 0.5857        | 0.8649        |
| MetaAc4C-noGAN | 80.94        | 81.80        | 80.09        | 0.6189        | 0.8651        |
| MetaAc4C       | 81.79        | 79.22        | 84.36        | 0.6368        | 0.8749        |
| STM-ac4C       | 82.01        | 82.44        | 81.58        | 0.6402        | 0.8804        |
| TBC-ac4C       | <b>84.48</b> | <b>83.30</b> | 85.65        | <b>0.6897</b> | <b>0.9080</b> |

The bold font denotes the optimal value obtained in each column

TABLE VI. COMPARATIVE EVALUATION OF MULTIPLE METHODS ON THE UNBALANCED DATASET

|                | ACC(%)       | Sn(%)        | Sp(%)        | MCC           | AUC           |
|----------------|--------------|--------------|--------------|---------------|---------------|
| PACES          | 79.26        | 7.28         | <b>95.49</b> | 0.0487        | 0.5139        |
| XG-ac4C        | 80.60        | 12.63        | 95.35        | 0.1280        | 0.5399        |
| DeepAc4C       | 80.51        | 79.65        | 81.37        | 0.6103        | 0.8640        |
| MetaAc4C-noGAN | 81.04        | 81.56        | 80.94        | 0.6209        | 0.8686        |
| MetaAc4C       | 82.87        | 80.94        | 84.80        | 0.6579        | 0.8951        |
| STM-ac4C       | <b>83.19</b> | 78.80        | 87.58        | <b>0.6664</b> | 0.8942        |
| TBC-ac4C       | 82.98        | <b>89.29</b> | 76.66        | 0.6649        | <b>0.8962</b> |

The bold font denotes the optimal value obtained in each column

#### ACKNOWLEDGMENT

\*\*\*

#### REFERENCES

- [1] P. Boccaletto, F. Stefaniak, A. Ray, A. Cappannini, S. Mukherjee, E. Purta et al., “MODOMICS: a database of RNA modification pathways,” *Nucleic Acids Res.*, vol. 50, pp. 231–235, January 2022.
- [2] F. Morena, C. Argentati, M. Bazzucchi, C. Emiliani, and S. Martino, “Above the epitranscriptome: RNA modifications and stem cell identity,” *Genes*, vol. 9, pp. 329, June 2018.
- [3] A. Roundtree, M. E. Evans, T. Pan, and C. He, “Dynamic RNA modifications in gene expression regulation,” *Cell*, vol. 169, pp. 1187–1200, June 2017.
- [4] V. A. Arzumanyan, G. V. Dolgalev, I. Y. Kurbatov, O. I. Kiseleva, and E. V. Poverennaya, “Epitranscriptome: review of top 25 most-studied RNA modifications,” *Int. J. Mol. Sci.*, vol. 23, pp. 13851, November 2022.
- [5] D. Arango, D. Sturgill, N. Alhusaini, A. A. Dillman, T. J. Sweet, G. Hanson et al., “Acetylation of cytidine in mRNA promotes translation efficiency,” *Cell*, vol. 175, pp. 1872–1886, December 2018.
- [6] G. Jin, M. Xu, M. Zou, and S. Duan, “The processing, gene regulation, biological functions, and clinical relevance of N4-Acetylcytidine on RNA: a systematic review,” *Mol. Ther. Nucleic Acids*, vol. 20, pp. 13–24, June 2020.
- [7] J. M. Thomas, C. A. Briney, K. D. Nance, J. E. Lopez, A. L. Thorpe, S. D. Fox et al., “A chemical signature for cytidine acetylation in RNA,” *J. Am. Chem. Soc.*, vol. 140, pp. 12667–12670, October 2018.
- [8] S. Ito, Y. Akamatsu, A. Noma, S. Kimura, K. Miyauchi, Y. Ikeuchi et al., “A single acetylation of 18 S rRNA is essential for biogenesis of the small ribosomal subunit in *Saccharomyces cerevisiae*,” *J. Bio. Chem.*, vol. 289, pp. 26201–26212, September 2014.
- [9] G. Thomas, J. Gordon, and H. Rogg, “N4-Acetylcytidine, a previously unidentified labile component of the small subunit of eukaryotic ribosomes,” *J. Biol. Chem.*, vol. 253, pp. 1101–1105, February 1978.
- [10] L. Xie, X. Zhong, W. Cao, J. Liu, X. Zu, and L. Chen, “Mechanisms of NAT10 as ac4C writer in diseases,” *Mol. Ther. Nucleic Acids*, vol. 32, pp. 359–368, April 2023.
- [11] X. Jiang, Y. Cheng, Y. Zhu, C. Xu, Q. Li, X. Xing et al., “Maternal NAT10 orchestrates oocyte meiotic cell-cycle progression and maturation in mice,” *Nat. Commun.*, vol. 14, pp. 3729, June 2023.
- [12] P. Bhargava, K. C. Fitzgerald, S. L. V. Venkata, M. D. Smith, M. D. Kornberg, E. M. Mowry et al., “Dimethyl fumarate treatment induces lipid metabolism alterations that are linked to immunological changes,” *Ann. Clin. Transl. Neurol.*, vol. 6, pp. 33–45, October 2018.
- [13] X. Zhang, J. Zeng, J. Wang, Z. Yang, S. Gao, H. Liu et al., “Revealing the potential markers of N(4)-acetylcytidine through acRIP-seq in triple-negative breast cancer,” *Genes*, vol. 13, pp. 2400, December 2022.
- [14] B. Li, D. Li, L. Cai, Q. Zhou, C. Liu, J. Lin et al., “Transcriptome-wide profiling of RNA N4-cytidine acetylation in *Arabidopsis thaliana* and *Oryza sativa*,” *Mol. Plant*, vol. 16, pp. 1082–1098, June 2023.
- [15] D. Stoeckle, M. Thellmann, and J. E. Vermeer, “Breakout-lateral root emergence in *Arabidopsis thaliana*,” *Curr. Opin. Plant Biol.*, vol. 41, pp. 67–72, February 2018.
- [16] R. David, A. Burgess, B. Parker, J. Li, K. Pulsford, T. Sibbritt et al., “Transcriptome-wide mapping of RNA 5-methylcytosine in *Arabidopsis* mRNAs and noncoding RNAs,” *Plant Cell*, vol. 29, pp. 445–460, March 2017.
- [17] W. Zhao, Y. Zhou, Q. Cui and Y. Zhou, “PACES: prediction of N4-acetylcytidine (ac4C) modification sites in mRNA,” *Sci. Rep.*, vol. 9, pp. 11112, July 2019.
- [18] W. Alam, H. Tayara, and K. T. Chong, “XG-ac4C: identification of N4-acetylcytidine (ac4C) in mRNA using eXtreme gradient boosting with electron-ion interaction pseudopotentials,” *Sci. Rep.*, vol. 10, pp. 20942, December 2020.
- [19] C. Wang, Y. Ju, Q. Zou, C. Lin, “DeepAc4C: a convolutional neural network model with hybrid features composed of physicochemical patterns and distributed representation information for identification of

- N4-acetylcytidine in mRNA,” *Bioinformatics*, vol. 38, pp. 52–57, December 2021.
- [20] W. Su, X. Q. Xie, X. W. Liu, D. Gao, C. Y. Ma et al., “iRNA-ac4C: a novel computational method for effectively detecting N4-acetylcytidine sites in human mRNA,” *Int. J. Biol. Macromol.*, vol. 227, pp. 1174–1181, February 2023.
- [21] F. L. Lai, and F. Gao, “LSA-ac4C: a hybrid neural network incorporating double-layer LSTM and self-attention mechanism for the prediction of N4-acetylcytidine sites in human mRNA,” *Int. J. Biol. Macromol.*, vol. 253, pp. 126837, December 2023.
- [22] J. Jia, X. Cao, and Z. Wei, “DLC-ac4C: a prediction model for N4-acetylcytidine sites in human mRNA based on DenseNet and bidirectional LSTM methods,” *Curr. Genomics*, vol. 24, pp. 171–186, November 2023.
- [23] N. T. Pham, A. T. Terrance, Y. J. Jeon, R. Rakkiyappan, and B. Manavalan, “ac4C-AFL: a high-precision identification of human mRNA N4-acetylcytidine sites based on adaptive feature representation learning,” *Mol. Ther. Nucleic Acids*, vol. 35, pp. 102192, April 2024.
- [24] J. Yuan, Z. Wang, Z. Pan, A. Li, Z. Zhang, and F. Cui, “DPNN-ac4C: a dual-path neural network with self-attention mechanism for identification of N4-acetylcytidine (ac4C) in mRNA,” *Bioinformatics*, vol. 40, btae625, November 2024.
- [25] W. He, Y. Han, Y. Zuo, and F. Guo, “NBCR-ac4C: A deep learning framework based on multivariate BERT for human mRNA N4-acetylcytidine sites prediction,” *J. Chem. Inf. Model.*, vol. 64, pp. 8074–8081, October 2024.
- [26] Z. Li, B. Jin, and J. Fang, “MetaAc4C: A multi-module deep learning framework for accurate prediction of N4-acetylcytidine sites based on pre-trained bidirectional encoder representation and generative adversarial networks,” *Genomics*, vol. 116, pp. 110749, January 2024.
- [27] M. Yi, F. Zhou, and Y. Deng, “STM-ac4C: a hybrid model for identification of N4-acetylcytidine (ac4C) in human mRNA based on selective kernel convolution, temporal convolutional network, and multi-head self-attention,” *Front. Genet.*, vol. 15, pp. 1408688, May 2024.
- [28] Y. Jia, Z. Zhang, S. Yan, Q. Zhang, L. Wei, and F. Cui, “Voting-ac4C: Pre-trained large RNA language model enhances RNA N4-acetylcytidine site prediction,” *Int. J. Biol. Macromol.*, vol. 282, pp. 136940, December 2024.
- [29] S. Zhang, K. Liu, and Y. Xu, “TransCNN: a novel architecture combining transformer and TextCNN for detecting N4-acetylcytidine sites in human mRNA,” *Anal. Bioche.*, vol. 703, pp. 115882, August 2025.
- [30] W. Li, and A. Godzik, “Cd-hit: a fast program for clustering and comparing large sets of protein or nucleotide sequences,” *Bioinformatics*, vol. 22, pp. 1658–1659, July 2006.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez et al., “Attention is all you need,” 2013, arXiv:1706.03762. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [32] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” 2018, arXiv:1803.01271. [Online]. Available: <https://arxiv.org/abs/1803.01271>
- [33] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training Recurrent Neural Networks,” 2012, arXiv:1211.5063. [Online]. Available: <https://arxiv.org/abs/1211.5063>
- [34] C. Li, Y. He, X. Li, and X. Jing, “BiGRU network for human activity recognition in high resolution range profile,” in 2019 International Radar Conference (RADAR), pp. 1–5, September 2019.
- [35] C. Chen, Q. Fan, and R. Panda, “CrossViT: cross-attention multi-scale vision transformer for image classification,” 2103, arXiv:2103.14899. [Online]. Available: <https://arxiv.org/abs/2103.14899>.
- [36] D.P. Kingma, and J. Ba, “Adam: a method for stochastic optimization,” 2014, arXiv:1412.6980. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [37] PyTorch: ReduceLROnPlateau PyTorch 1.9.0 documentation, [https://pytorch.org/docs/stable/generated/torch.optim.lr\\_scheduler.ReduceLROnPlateau.html](https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html).
- [38] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” 2019, arXiv:1907.10902. [Online]. Available: <https://arxiv.org/abs/1907.10902>
- [39] P. I. Frazier, “A tutorial on Bayesian optimization,” 2019, arXiv:1807.02811. [Online]. Available: <https://arxiv.org/abs/1807.02811>
- [40] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” 2018, arXiv:1802.03426. [Online]. Available: <https://arxiv.org/abs/1802.03426>